

The AI-coding landscape

For many years, programming often meant:

For many years, programming often meant:

- reading documentation

For many years, programming often meant:

- reading documentation
- searching Stack Overflow

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution
- adapting it until it worked

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution
- adapting it until it worked
- promising yourself you understood it

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution
- adapting it until it worked
- promising yourself you understood it

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution
- adapting it until it worked
- promising yourself you understood it

Large language models changed the rhythm of this work.

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution
- adapting it until it worked
- promising yourself you understood it

Large language models changed the rhythm of this work.

Now the first draft can arrive before you understand the problem.

For many years, programming often meant:

- reading documentation
- searching Stack Overflow
- copying a partial solution
- adapting it until it worked
- promising yourself you understood it

Large language models changed the rhythm of this work.

Now the first draft can arrive before you understand the problem.

That is powerful, but of course also dangerous.

Two visible shifts:

Two visible shifts:

- Stack Overflow question volume dropped sharply after ChatGPT became widely available.

Two visible shifts:

- Stack Overflow question volume dropped sharply after ChatGPT became widely available.
- Coding models have improved rapidly on longer and more complex tasks.

Two visible shifts:

- Stack Overflow question volume dropped sharply after ChatGPT became widely available.
- Coding models have improved rapidly on longer and more complex tasks.

Two visible shifts:

- Stack Overflow question volume dropped sharply after ChatGPT became widely available.
- Coding models have improved rapidly on longer and more complex tasks.

One way to measure this is **task completion time horizon**: how long a task can be while the model still succeeds at a given rate.

Two visible shifts:

- Stack Overflow question volume dropped sharply after ChatGPT became widely available.
- Coding models have improved rapidly on longer and more complex tasks.

One way to measure this is **task completion time horizon**: how long a task can be while the model still succeeds at a given rate.

Recent estimates suggest this horizon has been increasing roughly exponentially.

It is not useful to memorize which model is currently “best” – that will change.

The lesson

It is not useful to memorize which model is currently “best” – that will change.

What matters more:

It is not useful to memorize which model is currently “best” – that will change.

What matters more:

- what kinds of help these tools can provide

It is not useful to memorize which model is currently “best” – that will change.

What matters more:

- what kinds of help these tools can provide
- what kinds of mistakes they tend to make

It is not useful to memorize which model is currently “best” – that will change.

What matters more:

- what kinds of help these tools can provide
- what kinds of mistakes they tend to make
- how to structure your work so mistakes are caught

It is not useful to memorize which model is currently “best” – that will change.

What matters more:

- what kinds of help these tools can provide
- what kinds of mistakes they tend to make
- how to structure your work so mistakes are caught
- how to keep yourself in control of the scientific reasoning

Worked example: one Gemini call

Prompt:

Worked example: one Gemini call

Prompt:

```
write a Python function that returns Fisher's z transform of r
```

Worked example: one Gemini call

Prompt:

```
write a Python function that returns Fisher's z transform of r
```

This was sent to Gemini using the same REST pattern from the API chapter, with `GEMINI_API_KEY` read from the environment.

Gemini returned a good-looking implementation:

Gemini returned a good-looking implementation:

```
import math

def fisher_z_transform(r):
    """
    Calculates Fisher's z-transform of the correlation coefficient 'r'.
    """
    if not (-1 < r < 1):
        raise ValueError("Correlation coefficient must be between -1 and 1.")

    return math.atanh(r)
```

Gemini response, abbreviated

Gemini returned a good-looking implementation:

```
import math

def fisher_z_transform(r):
    """
    Calculates Fisher's z-transform of the correlation coefficient 'r'.
    """
    if not (-1 < r < 1):
        raise ValueError("Correlation coefficient must be between -1 and 1.")

    return math.atanh(r)
```

This is basically correct, but we are not done!

What should we check?

A good answer is still just a proposal.

What should we check?

A good answer is still just a proposal.

For the Fisher transform, obvious checks include:

What should we check?

A good answer is still just a proposal.

For the Fisher transform, obvious checks include:

- `fisher_z_transform(0) == 0`

What should we check?

A good answer is still just a proposal.

For the Fisher transform, obvious checks include:

- `fisher_z_transform(0) == 0`
- `fisher_z_transform(0.5)` is about `0.5493`

What should we check?

A good answer is still just a proposal.

For the Fisher transform, obvious checks include:

- `fisher_z_transform(0) == 0`
- `fisher_z_transform(0.5)` is about `0.5493`
- negative correlations give negative transformed values

What should we check?

A good answer is still just a proposal.

For the Fisher transform, obvious checks include:

- `fisher_z_transform(0) == 0`
- `fisher_z_transform(0.5)` is about `0.5493`
- negative correlations give negative transformed values
- `r = 1` and `r = -1` raise clear errors

What should we check?

A good answer is still just a proposal.

For the Fisher transform, obvious checks include:

- `fisher_z_transform(0) == 0`
- `fisher_z_transform(0.5)` is about `0.5493`
- negative correlations give negative transformed values
- `r = 1` and `r = -1` raise clear errors
- non-numeric inputs fail predictably

Try the same prompt again:

Try the same prompt again:

```
body = {  
  "contents": [{  
    "parts": [{  
      "text": "write a Python function that returns Fisher's z transform of r"  
    }]  
  }]  
}
```

Try the same prompt again:

```
body = {  
  "contents": [{  
    "parts": [{  
      "text": "write a Python function that returns Fisher's z transform of r"  
    }]  
  }]  
}
```

You may get a different implementation, that's expected!

LLMs remain controversial for good reasons:

LLMs remain controversial for good reasons:

- they can reproduce biases present in the corpus

LLMs remain controversial for good reasons:

- they can reproduce biases present in the corpus
- there are unresolved copyright and license questions

LLMs remain controversial for good reasons:

- they can reproduce biases present in the corpus
- there are unresolved copyright and license questions
- large-scale model training and inference use substantial energy and water

LLMs remain controversial for good reasons:

- they can reproduce biases present in the corpus
- there are unresolved copyright and license questions
- large-scale model training and inference use substantial energy and water
- they can produce confident falsehoods

LLMs remain controversial for good reasons:

- they can reproduce biases present in the corpus
- there are unresolved copyright and license questions
- large-scale model training and inference use substantial energy and water
- they can produce confident falsehoods

LLMs remain controversial for good reasons:

- they can reproduce biases present in the corpus
- there are unresolved copyright and license questions
- large-scale model training and inference use substantial energy and water
- they can produce confident falsehoods

The last point matters directly for coding! Wrong code often looks like right code.

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

- a function that does not exist

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

- a function that does not exist
- a parameter name from an older API

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

- a function that does not exist
- a parameter name from an older API
- a formula that is close to the right one, or an approximation

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

- a function that does not exist
- a parameter name from an older API
- a formula that is close to the right one, or an approximation
- a reference to a package that sounds plausible

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

- a function that does not exist
- a parameter name from an older API
- a formula that is close to the right one, or an approximation
- a reference to a package that sounds plausible

Hallucination

A hallucination is not just “making something up” – it is making something up **in the same fluent style** as true information. You lose the signal quality of fluency for competence.

In scientific computing this can look like:

- a function that does not exist
- a parameter name from an older API
- a formula that is close to the right one, or an approximation
- a reference to a package that sounds plausible

LLM hallucinations always looks good in context. As behavioral scientists we should probably call them “confabulations” instead.

Coding has one major advantage over many other uses of LLMs: we can run the result.

Coding has one major advantage over many other uses of LLMs: we can run the result.

Even better, we can write tests before accepting the result.

Coding has one major advantage over many other uses of LLMs: **we can run the result.**

Even better, **we can write tests before accepting the result.**

For scientific code, this seems like an excellent bargain between AI and human:

Coding has one major advantage over many other uses of LLMs: **we can run the result.**

Even better, **we can write tests before accepting the result.**

For scientific code, this seems like an excellent bargain between AI and human:

- AI can accelerate draft implementation.

Coding has one major advantage over many other uses of LLMs: **we can run the result.**

Even better, **we can write tests before accepting the result.**

For scientific code, this seems like an excellent bargain between AI and human:

- AI can accelerate draft implementation.
- Tests, review, and domain knowledge decide whether the draft survives.

Technical debt is the future cost of choices you make now: code that works today but will slow you down (or break) the next time you need to extend, port, or trust it.

Technical debt is the future cost of choices you make now: code that works today but will slow you down (or break) the next time you need to extend, port, or trust it.

Like financial debt, technical debt is not automatically bad – sometimes you borrow on purpose, to ship a result before a deadline or to keep a script readable while you're still exploring.

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

- Agents **mint debt very quickly**. A 200-line draft that runs can land in a repo in minutes.

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

- Agents **mint debt very quickly**. A 200-line draft that runs can land in a repo in minutes.
- Agents **hide local complexity**. Code that the author would not have written by hand is harder to keep in your head, which makes the next refactor (with or without AI) more expensive.

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

- Agents **mint debt very quickly**. A 200-line draft that runs can land in a repo in minutes.
- Agents **hide local complexity**. Code that the author would not have written by hand is harder to keep in your head, which makes the next refactor (with or without AI) more expensive.
- Borrowing is now nearly frictionless, but paying the debt requires a human to read the code!

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

- Agents **mint debt very quickly**. A 200-line draft that runs can land in a repo in minutes.
- Agents **hide local complexity**. Code that the author would not have written by hand is harder to keep in your head, which makes the next refactor (with or without AI) more expensive.
- Borrowing is now nearly frictionless, but paying the debt requires a human to read the code!
- Treat every accepted AI patch as a small loan.

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

- Agents **mint debt very quickly**. A 200-line draft that runs can land in a repo in minutes.
- Agents **hide local complexity**. Code that the author would not have written by hand is harder to keep in your head, which makes the next refactor (with or without AI) more expensive.
- Borrowing is now nearly frictionless, but paying the debt requires a human to read the code!
- Treat every accepted AI patch as a small loan.

AI tools change the interest rate on technical debt

AI assistants change the rate at which debt accrues:

- Agents **mint debt very quickly**. A 200-line draft that runs can land in a repo in minutes.
- Agents **hide local complexity**. Code that the author would not have written by hand is harder to keep in your head, which makes the next refactor (with or without AI) more expensive.
- Borrowing is now nearly frictionless, but paying the debt requires a human to read the code!
- Treat every accepted AI patch as a small loan.

The bottleneck in scientific programming is not the typing but the understanding!

In a generative workflow, the model produces output and waits.

In a generative workflow, the model produces output and waits.

This is an **open-loop** interaction:

In a generative workflow, the model produces output and waits.

This is an **open-loop** interaction:

human prompt -> model response -> human reads response -> next prompt

In a generative workflow, the model produces output and waits.

This is an **open-loop** interaction:

`human prompt -> model response -> human reads response -> next prompt`

Examples:

In a generative workflow, the model produces output and waits.

This is an **open-loop** interaction:

`human prompt -> model response -> human reads response -> next prompt`

Examples:

- asking a chatbot to explain code

In a generative workflow, the model produces output and waits.

This is an **open-loop** interaction:

`human prompt -> model response -> human reads response -> next prompt`

Examples:

- asking a chatbot to explain code
- asking for a function draft

In a generative workflow, the model produces output and waits.

This is an **open-loop** interaction:

`human prompt -> model response -> human reads response -> next prompt`

Examples:

- asking a chatbot to explain code
- asking for a function draft
- using autocomplete in an editor

In an agentic workflow, the model can use tools.

In an agentic workflow, the model can use tools.

This is closer to a **closed-loop** interaction:

In an agentic workflow, the model can use tools.

This is closer to a **closed-loop** interaction:

`generate code -> run tests -> read errors -> edit code -> run tests again`

In an agentic workflow, the model can use tools.

This is closer to a **closed-loop** interaction:

`generate code -> run tests -> read errors -> edit code -> run tests again`

The model is not just writing text, but is acting in a development environment and iterating on its own output.

Worked example: an agent loop

You can make your own agentic loop with a model API and a simple while loop.

Worked example: an agent loop

You can make your own agentic loop with a model API and a simple while loop.

```
while not tests_pass():  
    proposed_patch = model.generate_patch(  
        instructions=task,  
        context=current_files(),  
        last_error=test_output(),  
    )  
    apply_patch(proposed_patch)  
    run("python -m unittest discover -s tests")
```

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine
- break containment if you're not careful

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine
- break containment if you're not careful

Tool use changes the risk landscape

A chatbot can give you bad code but will sort of just sit there after.

An agent is much more powerful! It can...

- edit many files
- install dependencies
- run shell commands
- keep trying a bad approach until you stop it
- modify tests to make itself look successful
- delete or overwrite work
- **seriously** fuck up your codebase, not to mention your machine
- break containment if you're not careful

More autonomy → stronger guardrails.

- **Sandboxing:** isolate the agent from the rest of the system

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**
 - the agent can't rewrite git history

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**
 - the agent can't rewrite git history
- **Test constantly**

Basic safety measures: Sandboxing and permissions

- **Sandboxing:** isolate the agent from the rest of the system
 - sandboxed environments like Docker containers
- **Permissions:** restrict the agent's ability to read/write certain files
 - can't edit test suites
 - can't edit outside the project directory
 - can't read keys and other secrets
- **Commit early, commit often**
 - the agent can't rewrite git history
- **Test constantly**
 - the agent can still make a mess of your repo

The **Model Context Protocol (MCP)** is a standard protocol for connecting AI agents to tools.

The **Model Context Protocol** (MCP) is a standard protocol for connecting AI agents to tools.

The MCP documentation describes it as a “USB-C port for AI applications.”

The **Model Context Protocol** (MCP) is a standard protocol for connecting AI agents to tools.

The MCP documentation describes it as a “USB-C port for AI applications.”

That is a good metaphor:

The **Model Context Protocol** (MCP) is a standard protocol for connecting AI agents to tools.

The MCP documentation describes it as a “USB-C port for AI applications.”

That is a good metaphor:

- the model does not need a custom integration for every tool

The **Model Context Protocol** (MCP) is a standard protocol for connecting AI agents to tools.

The MCP documentation describes it as a “USB-C port for AI applications.”

That is a good metaphor:

- the model does not need a custom integration for every tool
- the tool exposes a standard interface

The **Model Context Protocol** (MCP) is a standard protocol for connecting AI agents to tools.

The MCP documentation describes it as a “USB-C port for AI applications.”

That is a good metaphor:

- the model does not need a custom integration for every tool
- the tool exposes a standard interface
- the agent can ask to use the tool when needed

An agent might use tools for:

An agent might use tools for:

- file system access

Example MCP tools

An agent might use tools for:

- file system access
- web search

An agent might use tools for:

- file system access
- web search
- database queries

An agent might use tools for:

- file system access
- web search
- database queries
- browser automation

An agent might use tools for:

- file system access
- web search
- database queries
- browser automation
- package documentation

An agent might use tools for:

- file system access
- web search
- database queries
- browser automation
- package documentation
- test execution

An agent might use tools for:

- file system access
- web search
- database queries
- browser automation
- package documentation
- test execution

An agent might use tools for:

- file system access
- web search
- database queries
- browser automation
- package documentation
- test execution

For web projects, a browser automation tool such as Playwright MCP can let an agent click through an application and inspect failures.

Example MCP tools

An agent might use tools for:

- file system access
- web search
- database queries
- browser automation
- package documentation
- test execution

For web projects, a browser automation tool such as Playwright MCP can let an agent click through an application and inspect failures.

MCPs add **productive constraint**.

The smallest MCP has three parts:

The smallest MCP has three parts:

1. A **server** advertises one or more **tools** (name, description, JSON schema for arguments).

The smallest MCP has three parts:

1. A **server** advertises one or more **tools** (name, description, JSON schema for arguments).
2. The **host** (your IDE or agent) offers those tools to the model.

The smallest MCP has three parts:

1. A **server** advertises one or more **tools** (name, description, JSON schema for arguments).
2. The **host** (your IDE or agent) offers those tools to the model.
3. The **interface**: the model returns a **tool call**; the server runs it and sends back a **result**.

The smallest MCP has three parts:

1. A **server** advertises one or more **tools** (name, description, JSON schema for arguments).
2. The **host** (your IDE or agent) offers those tools to the model.
3. The **interface**: the model returns a **tool call**; the server runs it and sends back a **result**.

The smallest MCP has three parts:

1. A **server** advertises one or more **tools** (name, description, JSON schema for arguments).
2. The **host** (your IDE or agent) offers those tools to the model.
3. The **interface**: the model returns a **tool call**; the server runs it and sends back a **result**.

The goal is to standardize tool discovery and invocation to avoid the possibility of hallucinations.

The smallest MCP has three parts:

1. A **server** advertises one or more **tools** (name, description, JSON schema for arguments).
2. The **host** (your IDE or agent) offers those tools to the model.
3. The **interface**: the model returns a **tool call**; the server runs it and sends back a **result**.

The goal is to standardize tool discovery and invocation to avoid the possibility of hallucinations.

You make a **schema** to describe the tool's arguments and return value. The host can validate arguments **before** running anything.

Example: Tool definition

Suppose an MCP server exposes exactly one tool: **d_prime**.

Example: Tool definition

Suppose an MCP server exposes exactly one tool: **d_prime**.

```
{ "name": "d_prime",  
  "description": "SDT d' from outcome counts.",  
  "inputSchema": {  
    "type": "object",  
    "properties": {  
      "hits": {"type": "integer", "description": "N hits"},  
      "false_alarms": {"type": "integer", "description": "N false alarms"},  
      "n_signal": {"type": "integer", "description": "N signals"},  
      "n_noise": {"type": "integer", "description": "N non-signals"}  
    },  
    "required": ["hits", "false_alarms", "n_signal", "n_noise"] } }
```


The user asks: “Given 7 hits out of 10 signals and 2 false alarms out of 10 non-signals, what is d' ?”

Models issue tool calls

The user asks: “Given 7 hits out of 10 signals and 2 false alarms out of 10 non-signals, what is d' ?”

The model asks the host to call `d_prime` with concrete counts:

Models issue tool calls

The user asks: “Given 7 hits out of 10 signals and 2 false alarms out of 10 non-signals, what is d' ?”

The model asks the host to call `d_prime` with concrete counts:

```
{ "name": "d_prime",  
  "arguments": {  
    "hits": 7,  
    "false_alarms": 2,  
    "n_signal": 10,  
    "n_noise": 10  
  } }
```

Models issue tool calls

The user asks: “Given 7 hits out of 10 signals and 2 false alarms out of 10 non-signals, what is d' ?”

The model asks the host to call `d_prime` with concrete counts:

```
{ "name": "d_prime",  
  "arguments": {  
    "hits": 7,  
    "false_alarms": 2,  
    "n_signal": 10,  
    "n_noise": 10  
  } }
```

Same tool can be used by any MCP client that speaks the protocol.

The server computes d' from $H = 7/10$ and $F = 2/10$ and returns a structured result:

The server computes d' from $H = 7/10$ and $F = 2/10$ and returns a structured result:

```
{  
  "content": [  
    { "type": "text", "text": "0.5244..." }  
  ]  
}
```

The server computes d' from $H = 7/10$ and $F = 2/10$ and returns a structured result:

```
{  
  "content": [  
    { "type": "text", "text": "0.5244..." }  
  ]  
}
```

The model reads that result and writes the final answer to the user.

The server computes d' from $H = 7/10$ and $F = 2/10$ and returns a structured result:

```
{  
  "content": [  
    { "type": "text", "text": "0.5244..." }  
  ]  
}
```

The model reads that result and writes the final answer to the user.

Official overview: modelcontextprotocol.io.

Servers answer tool calls

The server computes d' from $H = 7/10$ and $F = 2/10$ and returns a structured result:

```
{  
  "content": [  
    { "type": "text", "text": "0.5244..." }  
  ]  
}
```

The model reads that result and writes the final answer to the user.

Official overview: modelcontextprotocol.io.

(We'll soon talk about how to wire MCPs up for Cursor.)

Example: runnable server in this repo

The same `d_prime` tool is implemented with the official Python SDK:

Example: runnable server in this repo

The same `d_prime` tool is implemented with the official Python SDK:

- Server: `files/mcp_sdt_server/sdt_server.py` (FastMCP, stdio transport)

Example: runnable server in this repo

The same `d_prime` tool is implemented with the official Python SDK:

- Server: `files/mcp_sdt_server/sdt_server.py` (FastMCP, stdio transport)
- Smoke-test client: `files/mcp_sdt_server/client_demo.py`

Example: runnable server in this repo

The same `d_prime` tool is implemented with the official Python SDK:

- Server: `files/mcp_sdt_server/sdt_server.py` (FastMCP, stdio transport)
- Smoke-test client: `files/mcp_sdt_server/client_demo.py`
- Setup: `files/mcp_sdt_server/README.md`

Example: runnable server in this repo

The same `d_prime` tool is implemented with the official Python SDK:

- Server: `files/mcp_sdt_server/sdt_server.py` (FastMCP, stdio transport)
- Smoke-test client: `files/mcp_sdt_server/client_demo.py`
- Setup: `files/mcp_sdt_server/README.md`

Example: runnable server in this repo

The same `d_prime` tool is implemented with the official Python SDK:

- Server: `files/mcp_sdt_server/sdt_server.py` (FastMCP, stdio transport)
- Smoke-test client: `files/mcp_sdt_server/client_demo.py`
- Setup: `files/mcp_sdt_server/README.md`

After `pip install -r requirements.txt` in a venv, `python client_demo.py` should print `tools: ['d_prime']` and the `d'` value for 7/10 hits and 2/10 false alarms.

Four interaction patterns

Pattern	Typical use
Single-turn generation	Draft one function, regular expression, SQL query, or explanation
Context-aware completion	Fill in code inside an editor using nearby files as context
Programmatic access	Call a model API from Python, as in the Gemini example
Autonomous execution	Let an agent inspect files, edit code, run tests, and revise

Four interaction patterns

Pattern	Typical use
Single-turn generation	Draft one function, regular expression, SQL query, or explanation
Context-aware completion	Fill in code inside an editor using nearby files as context
Programmatic access	Call a model API from Python, as in the Gemini example
Autonomous execution	Let an agent inspect files, edit code, run tests, and revise

These are not strict categories, most current tools blur them.

Pattern 1: single-turn generation

- You ask for something once.

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

- ambiguous scientific logic

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

- ambiguous scientific logic
- multi-file refactors

Pattern 1: single-turn generation

- You ask for something once.
- The model answers once.

Good for:

- small examples
- explanations
- scaffolding
- syntax you half-remember

Bad for:

- ambiguous scientific logic
- multi-file refactors
- tasks where correctness is not easy to check

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby
- the next step is local and obvious

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby
- the next step is local and obvious

Pattern 2: context-aware completion

- The model reads nearby code and predicts what should come next.
- This can feel like pair programming.

It is useful when:

- naming conventions are already clear
- types and function signatures are nearby
- the next step is local and obvious

It is risky when the local pattern is wrong – AI is **great** at imitating bad examples. It's also **wildly annoying** when it keeps trying to autocomplete based on some misunderstanding.

Pattern 3: programmatic access

You call the model from code. That lets you:

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops
- combine model output with automated checks

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops
- combine model output with automated checks

Pattern 3: programmatic access

You call the model from code. That lets you:

- repeat prompts
- compare outputs
- save responses
- build retry loops
- combine model output with automated checks

A model API is still just an API!

Pattern 4: autonomous execution

- You give the agent a goal.

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

- weakly specified scientific methods

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

- weakly specified scientific methods
- code with no tests

Pattern 4: autonomous execution

- You give the agent a goal.
- It explores, edits, runs commands, and reports back.

Good for:

- well-tested refactors
- mechanical code changes
- repetitive cleanup
- generating documentation from existing structure

Risky for:

- weakly specified scientific methods
- code with no tests
- sensitive data

Four interaction patterns

Pattern	Typical use
Single-turn generation	Draft one function, regular expression, SQL query, or explanation
Context-aware completion	Fill in code inside an editor using nearby files as context
Programmatic access	Call a model API from Python, as in the Gemini example
Autonomous execution	Let an agent inspect files, edit code, run tests, and revise

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

The core shift

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success
- reviewing code

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success
- reviewing code
- testing behavior

The core shift

AI does not make programming expertise obsolete, but changes the balance of which skills matter.

Less important:

- remembering every syntax detail
- writing boilerplate by hand
- starting from a blank file

More important:

- decomposing the problem
- defining success
- reviewing code
- testing behavior
- recognizing when the model is going in circles